
PWNAudit

NetScope

Wireshark-style live packet analyzer for Windows
with Wi-Fi 2.4 GHz / 5 GHz visibility, attack-detection rules, and an authorised
pentest module.

USER GUIDE · Version 0.1.0

PWNAudit · pwnaudit.com

Contents

1.	Introduction	3
2.	Installation	4
3.	Main interface — sidebar navigation	5
4.	Tab 1 · Interfaces (live capture)	6
5.	Tab 2 · Wireless (2.4 / 5 GHz Wi-Fi)	8
6.	Capture view — tri-pane analysis	10
7.	Tab 3 · Rules — attack detection	12
8.	Tab 4 · Pentest — authorised testing	14
9.	Architecture — how it works inside	17
10.	Troubleshooting & FAQ	18
11.	Credits & licence	20

1. Introduction

PWNAudit NetScope is a Windows desktop application for live network packet capture and analysis, in the style of Wireshark. Unlike Wireshark, NetScope was built specifically to make the **2.4 GHz vs 5 GHz Wi-Fi** story visible at a glance — useful when you are investigating throughput, coverage, or interference on your own network.

What you can do with it:

- See every Wi-Fi adapter on your machine, the SSID it is connected to, the band (2.4 GHz / 5 GHz), the channel and the live signal strength.
- Capture every packet flowing across any local network adapter (Wi-Fi, Ethernet, VPN, virtual).
- Run live attack-detection rules against the capture stream — SYN floods, port scans, ARP spoofing, deauth bursts.
- Author your own detection rules using a small JSON DSL — no code required.
- Run authorised pentest tools (port scanner, ARP scan, ARP spoof, 802.11 deauth, beacon flood) for testing your own network or a lab environment.

Important: NetScope is a defensive / educational tool. All traffic stays on your machine — nothing is uploaded. Run the pentest modules only against networks you own or have explicit written permission to test.

2. Installation

System requirements:

- Windows 10 or 11 (64-bit)
- Administrator rights (the app self-elevates via UAC)
- ≈ 60 MB free disk, 200 MB RAM during capture
- **Npcap** packet-capture driver (free, separate install)

Step 1 — Install Npcap

Download from <https://npcap.com/#download> and run the installer. Use the default options — NetScope works with both WinPcap-compatible mode and the modern driver.

Step 2 — Download NetScope

Open the PWNAudit dashboard at pwnaudit.com/dashboard/netscope and click **Download for Windows**. The .exe is hosted on GitHub Releases.

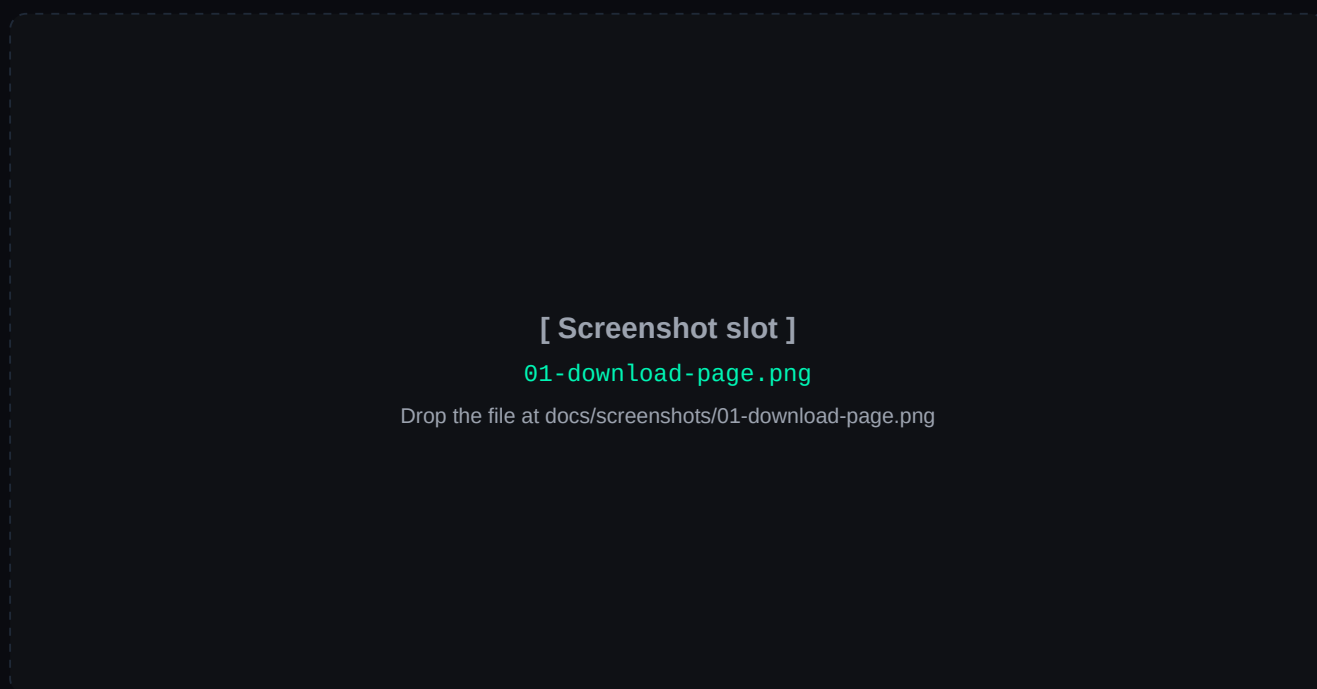


Figure 2.1 — The NetScope download page on the PWNAudit dashboard.

Step 3 — Run the installer

Double-click `PWNAudit-NetScope-Setup.exe`, approve the UAC prompt, and click through the wizard. After Finish, the app shows up under **PWNAudit** in Windows Search.

Windows SmartScreen may show a warning the first time you run an unsigned executable. Click **More info** → **Run anyway**. This warning disappears once the binary builds reputation or is code-signed.

3. Main interface

When you launch NetScope you land on the main window. The left rail is the navigation sidebar; the right side is the active page.

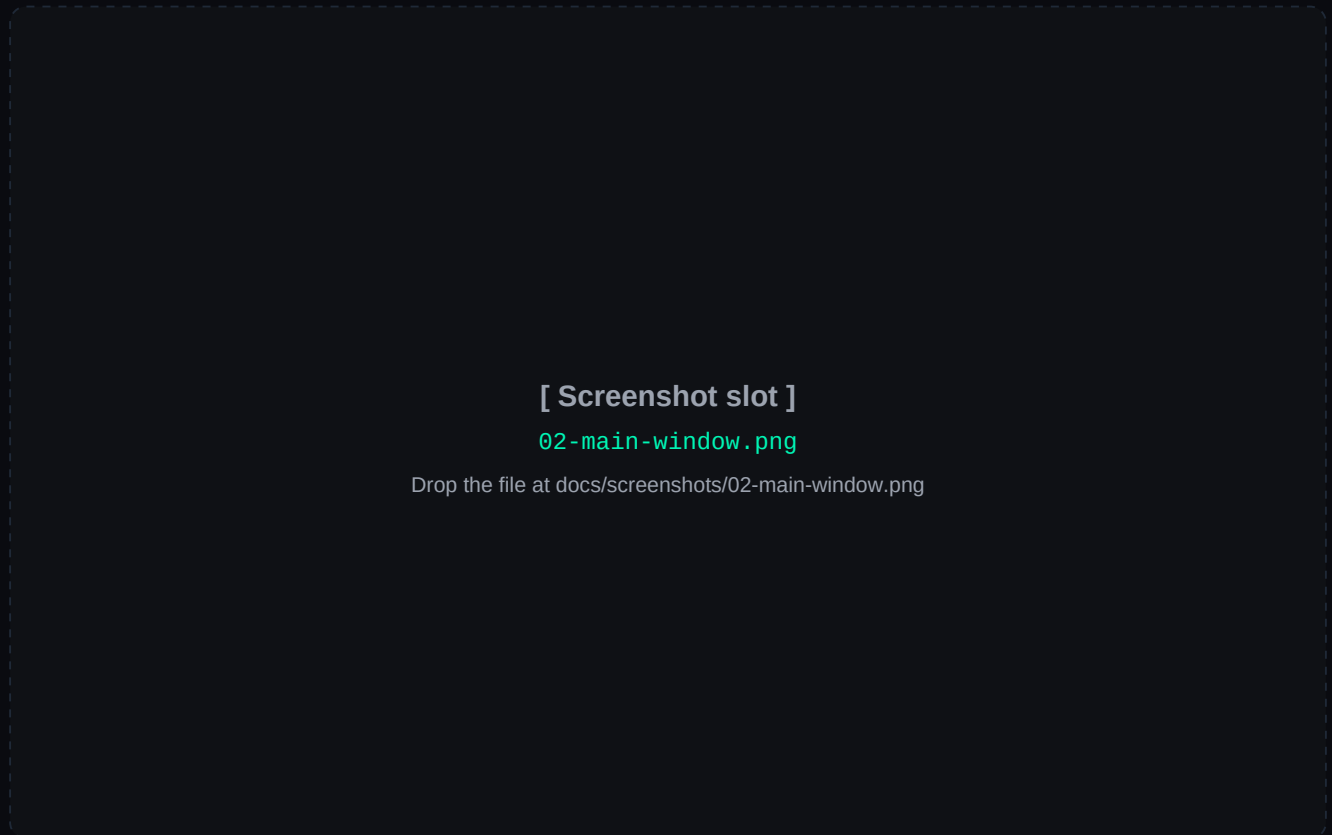


Figure 3.1 — Main window with sidebar navigation and the Interfaces page.

The four tabs:

- **Interfaces** — every local network adapter with live throughput sparklines. Click to start a generic capture.
- **Wireless** — Wi-Fi-aware view: connected SSID, BSSID, channel, band (2.4 / 5 GHz), nearby networks, capture launcher.
- **Rules** — attack-detection rules and live alerts.
- **Pentest** — authorised attack modules (port scan, ARP spoof, deauth, ...). Gated behind a confirmation banner.

4. Tab 1 · Interfaces

The Interfaces tab is the default landing page. It lists every network adapter that `psutil` reports — Wi-Fi, Ethernet, VPN tunnels, virtual adapters, even Bluetooth PAN.

Each row shows:

- Type glyph: ■ Wi-Fi · ■ Ethernet · ■ VPN · ■ other
- Adapter name and IP address
- Live **sparkline** of packets-per-second over the last 60 seconds — pulled directly from `psutil.net_io_counters()`, so the numbers match Windows Task Manager
- Current packets/sec rate

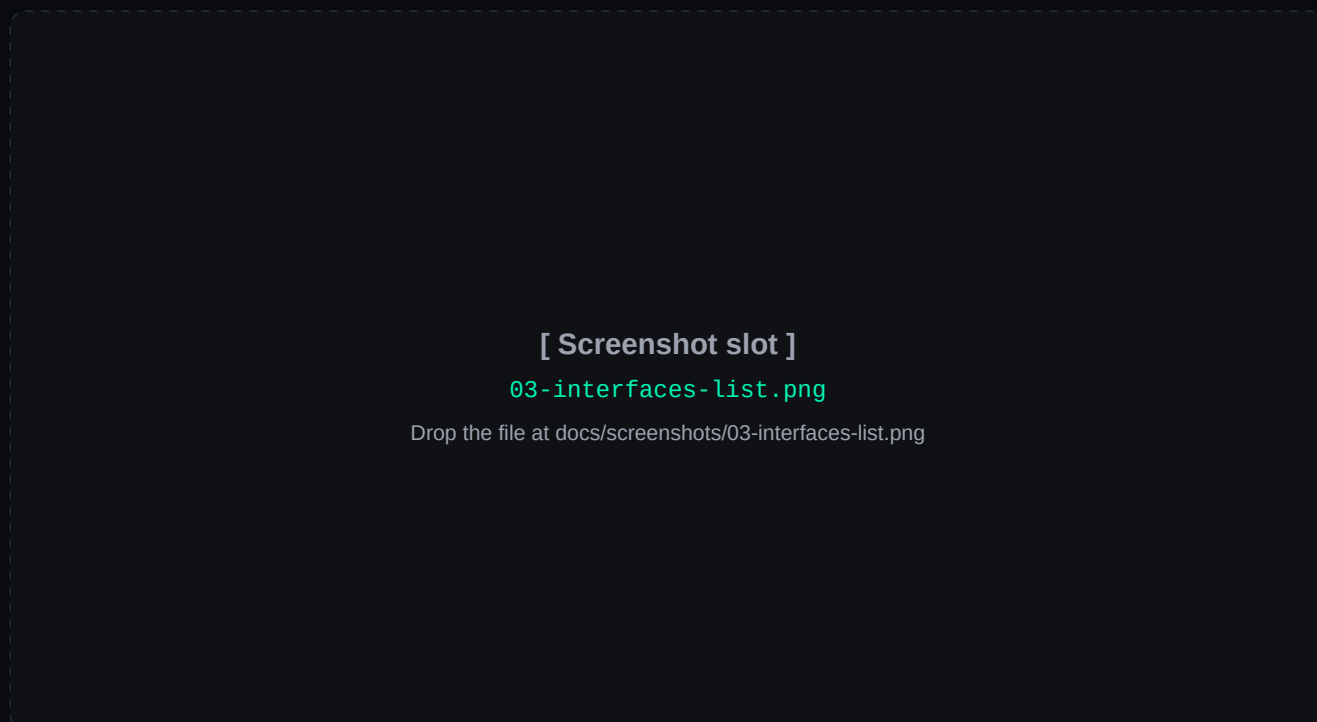


Figure 4.1 — Interfaces list with live sparklines. The accent-coloured row is what your machine is currently sending traffic over.

BPF capture filter

The text box at the bottom accepts a kernel-level **BPF filter** (same syntax as Wireshark / `tcpdump`).

Examples:

```
tcp port 80
host 192.168.1.10
udp and port 53
tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0
```

Clicking any interface row launches a capture session with the filter applied. The view switches to the **Capture page** (see section 6).

5. Tab 2 · Wireless

The Wireless tab is the headline feature for the 2.4 GHz / 5 GHz use case. It pulls live state from `netsh wlan show interfaces` and renders it as labelled tiles so you can read SSID / band / signal at a glance.

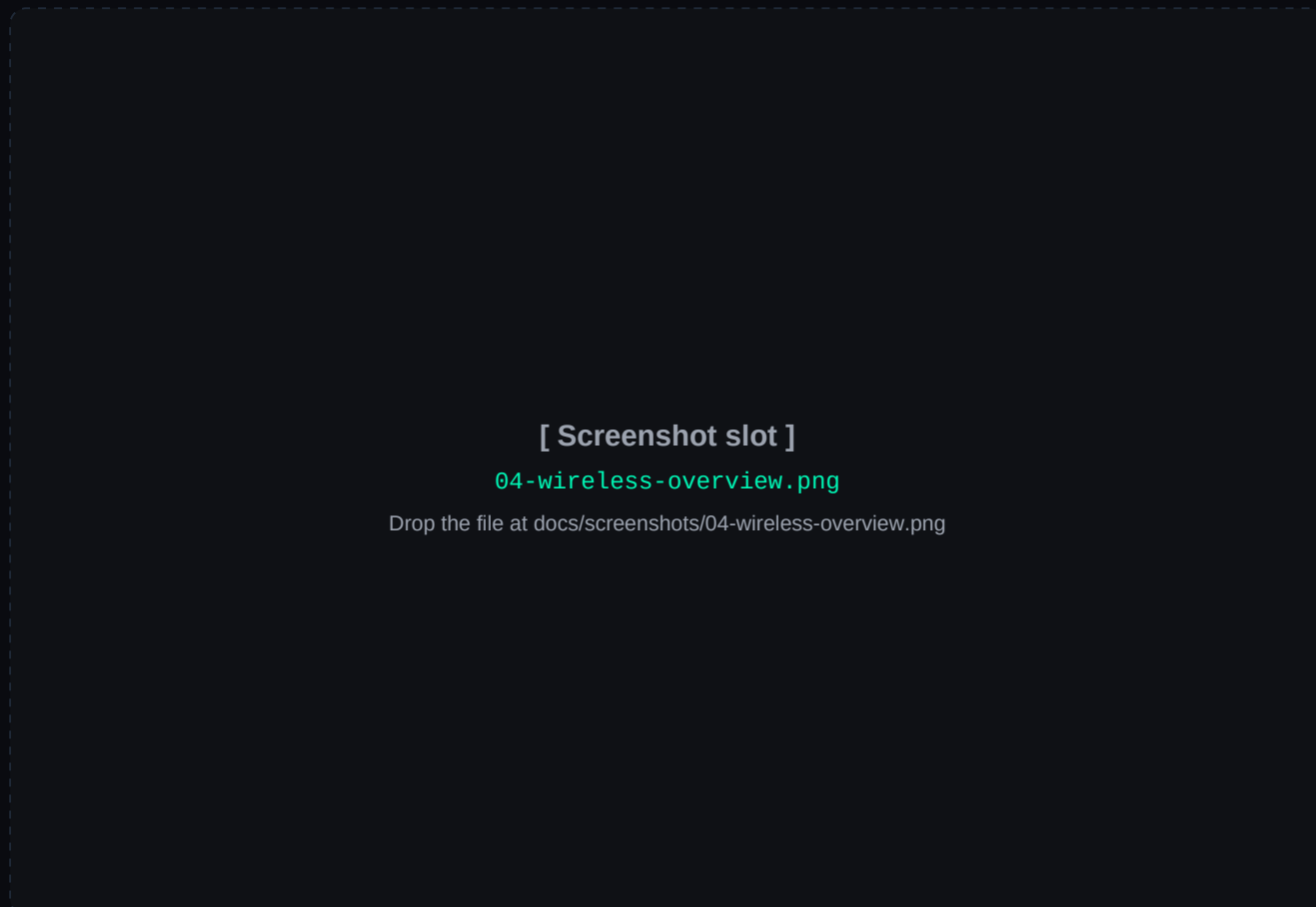


Figure 5.1 — Wireless tab showing the connected adapter on 5 GHz (channel 36) and a list of every nearby BSSID with its band.

Live status tiles

- **Connected SSID** — the network you are joined to
- **BSSID** — the MAC of the AP radio you are talking to
- **Channel** — IEEE 802.11 channel number
- **Band** — derived from the channel: 1–14 → 2.4 GHz, 32–177 → 5 GHz
- **Signal** — bar chart and percentage from the OS
- **Radio · Link** — 802.11 standard (ax / ac / n) and the current Rx / Tx negotiated rate

Visible networks list

Below the tiles is every BSSID the radio currently sees, sorted by signal strength. Use the **Band** dropdown above the list to filter to just 2.4 GHz, 5 GHz, or 6 GHz APs.

[Screenshot slot]

05-wireless-band-filter.png

Drop the file at docs/screenshots/05-wireless-band-filter.png

Figure 5.2 — Filtering the list to only 5 GHz. Useful when investigating channel congestion on a single band.

Launching capture

Click any row to select that BSSID, then hit **Capture on this band** — NetScope starts a session with the BPF filter `ether host <BSSID>` pre-applied, so you see only the traffic for that one AP.

Note: Without monitor mode (which most consumer Wi-Fi cards on Windows don't support via Npcap), the capture sees Ethernet-style frames the host is part of — beacons, probes, and deauths from third-party devices require an AirPcap or a monitor-mode-capable USB adapter (e.g. Alfa AWUS036ACH with the right driver).

6. Capture view

Once a capture is running, the view switches to a Wireshark-style tri-pane layout: packet list at the top, protocol tree on the bottom-left, hex+ASCII dump on the bottom-right.

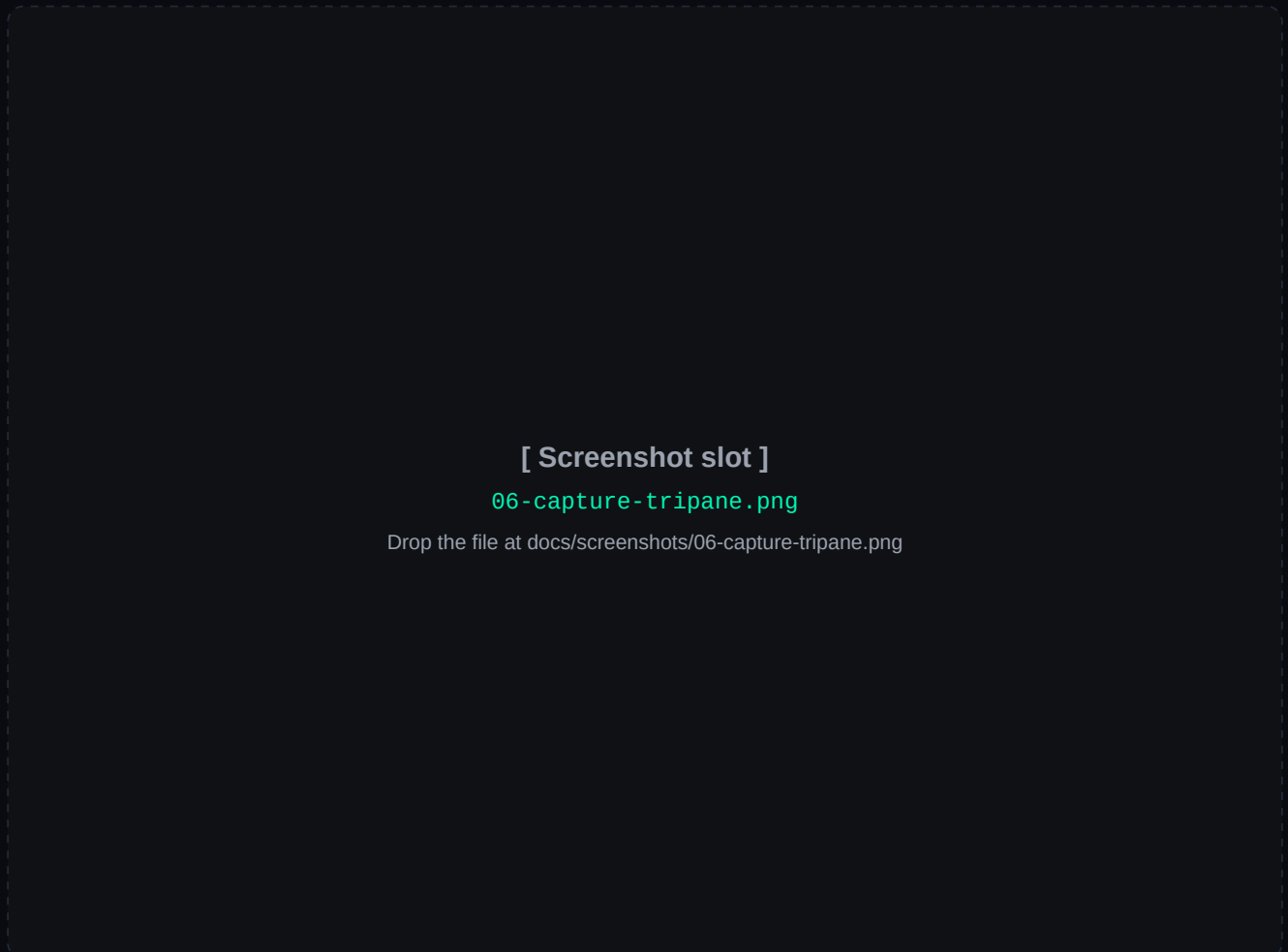


Figure 6.1 — Live capture session. Selecting a packet expands its protocol stack and renders the raw bytes.

Toolbar (top)

- **← Interfaces** — stop and return to the previous tab
- **Display filter** — substring / IP / port / proto match (debounced 250 ms)
- **■ Stop / ■ Resume** — pause and resume capture without losing already-captured packets
- **Clear** — drop all in-memory packets (frees RAM during long sessions)

Packet list

The table is virtualized via `QAbstractTableModel` and can hold up to 200 000 packets in memory before eviction. Columns: # · time · src · dst · proto · length · info.

Status bar (bottom)

- Capturing/Stopped indicator with the active interface name
- Total packet count

- Live **pps** (packets per second) over the last 500 ms window

Keyboard shortcuts:

```
Ctrl+E – toggle capture (stop / resume)
Ctrl+L – clear the packet list
Ctrl+F – focus the display-filter box
```

7. Tab 3 · Rules — attack detection

The Rules tab is a small intrusion-detection engine that evaluates every captured packet against a list of rules. When a sliding-window threshold is crossed an **Alert** fires — shown as a toast in the lower-right corner and as a Windows tray balloon when the app is in the background.

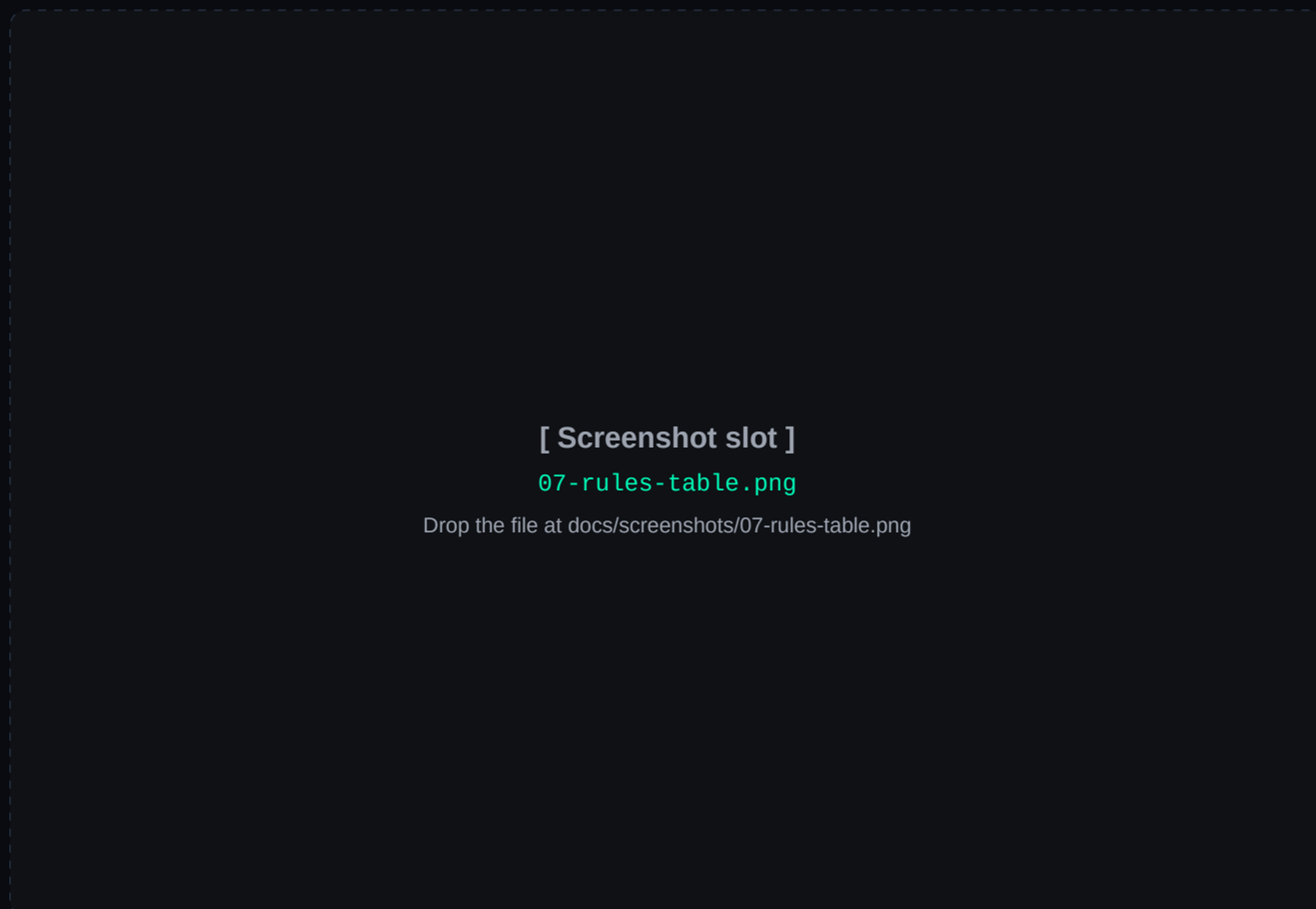


Figure 7.1 — Rules tab. The toggle on each row enables / disables the rule. The bottom panel streams alerts in real time.

Built-in rules (shipped with v0.1.0)

Rule	Severity	Threshold	What it catches
builtin.syn-flood	high	200 / 10s	Half-open SYN spam from one source — DoS attempt
builtin.port-scan	medium	30 / 10s	Many distinct ports from one IP — nmap-style scan
builtin.dns-flood	low	80 / 10s	Excessive DNS queries — DGA malware indicator
builtin.arp-spoof	high	20 / 10s	Unsolicited ARP replies — possible MITM
builtin.icmp-flood	medium	200 / 10s	Ping flood / DoS
builtin.wifi-deauth	high	10 / 5s	802.11 deauth bursts (monitor mode required)
builtin.wifi-beacon-flood	medium	80 / 10s	Spoofed AP beacons (monitor mode required)

Writing a custom rule

Click **+ New rule** to open the rule editor. Rules are described in a small JSON DSL — leave any field blank to wildcard it.

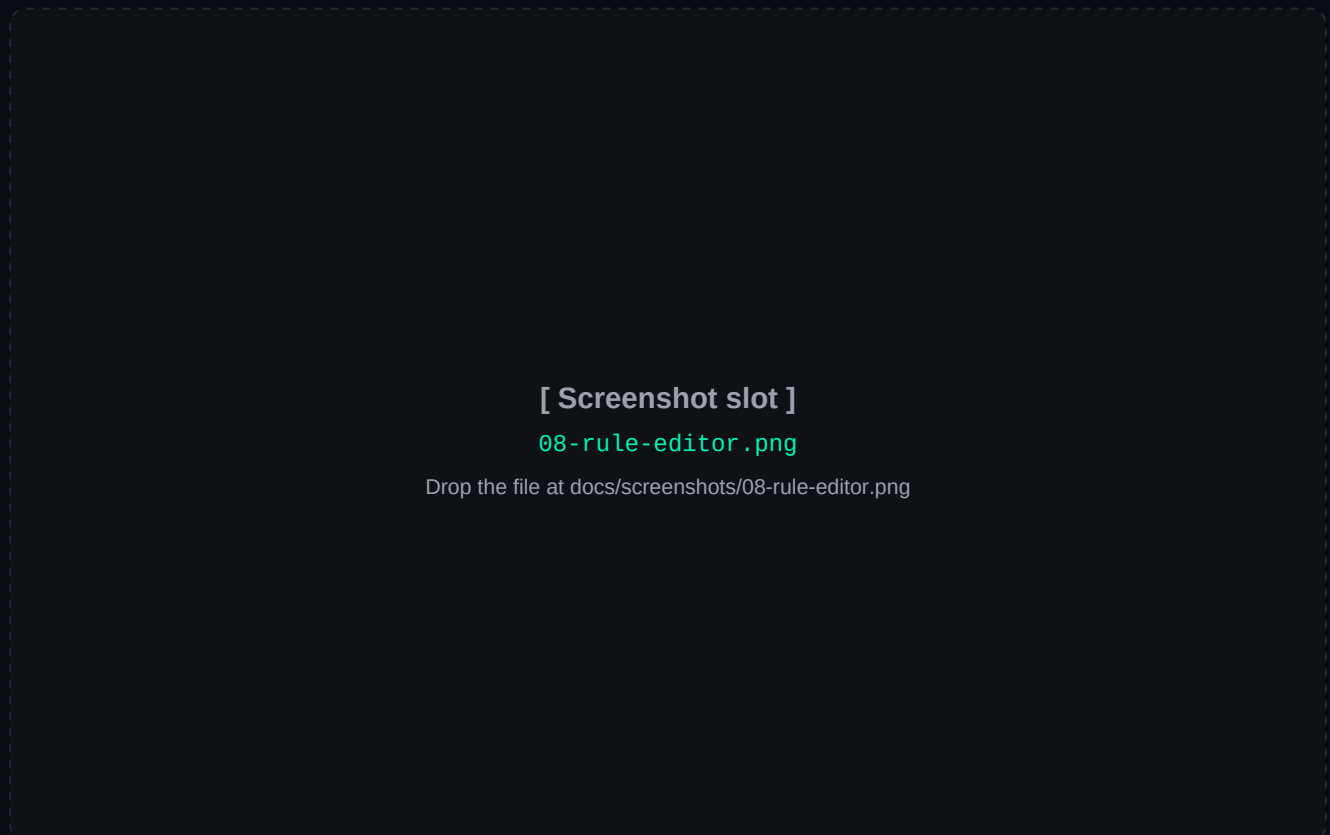


Figure 7.2 — Rule editor dialog. The example creates a rule that fires when more than 100 DNS queries hit 8.8.8.8 in 60 seconds from any one source.

DSL fields:

```
protocol = TCP | UDP | ICMP | ARP | DNS | DHCP | Dot11
src_ip = "any" | "10.0.0.1" | "192.168.1.0/24"
dst_ip = "any" | "8.8.8.8" | "10.0.0.0/24"
dst_port = null | 22 | 443
tcp_flags = null | "S" | "SA" | "FA"
key_by = src_ip | src_mac | dst_ip | dst_port | none
threshold = number of matches
window_sec = sliding window in seconds
```

Custom rules are persisted at %APPDATA%\PWNAudit-NetScope\rules.json and reloaded on every app launch. Built-in rules can be enabled/disabled but not edited — disable them and create a custom variant if you need different thresholds.

8. Tab 4 · Pentest — authorised testing

The Pentest tab contains active testing tools. All five modules are gated behind a red authorisation banner — the Run buttons stay disabled until you tick the confirmation checkbox.

Authorised use only. Run these modules only against networks you own or have explicit written permission to test. Running attacks against networks you do not own is illegal in most jurisdictions.

[Screenshot slot]

09-pentest-overview.png

Drop the file at docs/screenshots/09-pentest-overview.png

Figure 8.1 — Pentest tab with the authorisation banner unticked (Run buttons disabled).

Module 1 — TCP port scan

Touches each port on a target host with a connect()-style probe. Open ports are streamed into the attack log.

- **Target:** hostname or IP (DNS-resolved)
- **Ports:** comma list and ranges, e.g. 22, 80, 443, 8000-8010

[Screenshot slot]

10-pentest-portscan.png

Drop the file at docs/screenshots/10-pentest-portscan.png

Figure 8.2 — Port scan against 192.168.1.10 finding SSH and HTTP/S open.

Module 2 — ARP host discovery

Sweeps a /24 (or smaller) with ARP **who-has** and lists every MAC that responds. Hard cap of 1 024 hosts to prevent accidental scans of huge networks.

Module 3 — ARP spoof (MITM test)

Forges ARP replies so the victim learns the gateway is at your MAC. Used to verify your own IDS / endpoint protection notices the attack.

Stop & restore always sends five corrected ARP replies before quitting, so the victim's ARP cache is repaired. Don't force-quit the app while spoofing — the cache stays poisoned for ~30 seconds otherwise.

Modules 4-5 — Wi-Fi (deauth, beacon flood)

These require **monitor mode** with packet injection. Most consumer Wi-Fi cards on Windows don't support this without specialised drivers. If your adapter rejects the injection, the log says exactly that:

```
[deauth] sendp failed: ...  
This usually means the adapter does not support  
monitor-mode injection on Windows.
```

Adapters that work for monitor-mode injection on Windows include the Alfa AWUS036ACH (with the right Realtek driver) and most TP-Link cards re-flashed with the Atheros stack.

Attack log

The right-side panel records every action with a wall-clock timestamp. Useful for school write-ups: at the end of a session click **Clear log** only after you've copied the relevant lines into your report.

[Screenshot slot]

11-pentest-log.png

Drop the file at docs/screenshots/11-pentest-log.png

Figure 8.3 — Attack log with port-scan and ARP-spoof activity.

9. Architecture

Tech stack:

- **Python 3.11** + **PyQt6** for the desktop UI
- **scapy** for protocol dissection and packet injection
- **Npcap** as the underlying capture driver
- **psutil** for live interface counters
- **PyInstaller** + **Inno Setup** for the Windows installer

Threading model:

Capture happens on a background thread (scapy's AsyncSniffer). Packets land in a lock-protected buffer; a 100 ms QTimer drains the buffer into a Qt model on the UI thread. The rule engine evaluates each packet on the sniffer thread and emits alerts back to the UI via `QTimer.singleShot(0, ...)`.

File layout:

```
netscope/  
■ app.py – main window + sidebar nav  
■ welcome.py – Interfaces tab + sparklines  
■ wireless_page.py – Wireless tab (2.4 / 5 GHz)  
■ wifi.py – netsh parser, band detection  
■ capture.py – AsyncSniffer wrapper  
■ dissect.py – protocol summarisation  
■ model.py – Qt table model + filter proxy  
■ rules.py – rules engine + built-in rules  
■ rules_page.py – Rules tab UI  
■ notify.py – toast + tray notifications  
■ pentest.py – attack module backends  
■ pentest_page.py – Pentest tab UI  
■ interfaces.py – adapter enumeration  
■ theme.py – dark-theme QSS
```

10. Troubleshooting & FAQ

Npcap is required dialog appears every launch

Npcap is not installed or its DLL was renamed. Install / reinstall from npcap.com. After installation, restart NetScope.

UAC prompt is denied

NetScope requires admin rights for raw packet access. If you cancel the UAC prompt the app exits. Right-click PWNAudit-NetScope.exe → Run as administrator.

Wireless tab shows "No Wi-Fi adapter"

Your machine has no Wi-Fi radio or Windows has it disabled. Plug in a USB Wi-Fi dongle or enable the internal radio in Settings → Network & Internet → Wi-Fi, then click Refresh.

802.11 deauth says "sendp failed"

The adapter doesn't support monitor-mode injection via Npcap on Windows. Use a compatible adapter (Alfa AWUS036ACH, TP-Link TL-WN722N v1) or run from a Linux box with airmmon-ng.

Windows Defender flags the .exe as unknown

The binary is not yet code-signed. Click "More info" → "Run anyway". After enough downloads, SmartScreen builds reputation and stops warning.

Custom rules don't survive across app restarts

Check that %APPDATA%\PWNAudit-NetScope\ exists and is writable. If you renamed the app, the config dir name changes — copy rules.json across manually.

Capture drops packets at very high rates (>100k pps)

The Python+scapy pipeline tops out around 100-150k pps on a modern CPU. For higher rates use Wireshark with a kernel-mode capture filter to pre-filter at the driver layer.

11. Credits & licence

Open-source dependencies:

- **scapy** — GPLv2 (<https://scapy.net>)
- **PyQt6** — GPLv3 / commercial
- **psutil** — BSD
- **Npcap** — proprietary, free for personal use
- **Inno Setup** — modified BSD
- **reportlab** — BSD (this guide)

Project:

PWNAudit NetScope source code: github.com/vagifmammadov/pwnaudit-netscope

PWNAudit dashboard: pwnaudit.com

End of guide. Generated with `docs/generate_guide.py`.